

CONTENTS OF THE CHAPTER

- *Basic Concepts of Object-Oriented Programming: Classes, Objects, and Attributes*
- *Constructor Methods*
- *Getter and Setter Methods*
- *Object Comparison*
- *Arrays of Objects*
- *Overview of Common Built-in Classes in Java*
- *Inheritance, Polymorphism, and Abstraction in Object-Oriented Programming*

Chapter 5 addresses a crucial aspect of Java programming: object-oriented programming. This chapter presents the fundamental concepts of object-oriented programming, including classes, attributes, objects, and constructors. It also discusses the core principles of object-oriented programming—inheritance, abstraction, polymorphism, and encapsulation—to help readers understand how object-oriented programming is applied in Java.

5.1. OBJECT-ORIENTED PROGRAMMING

5.1.1. Basic Concepts

Object-Oriented Programming (OOP) is an application development method based on the concepts of **classes** and **objects** :

- **Object:** An entity that has state and behavior. Examples include bicycles, tables, chairs, etc. An object can represent either a physical or a logical entity.
- **Class:** A blueprint or template for creating objects. It is a logical entity that defines the properties and behaviors shared by its objects.

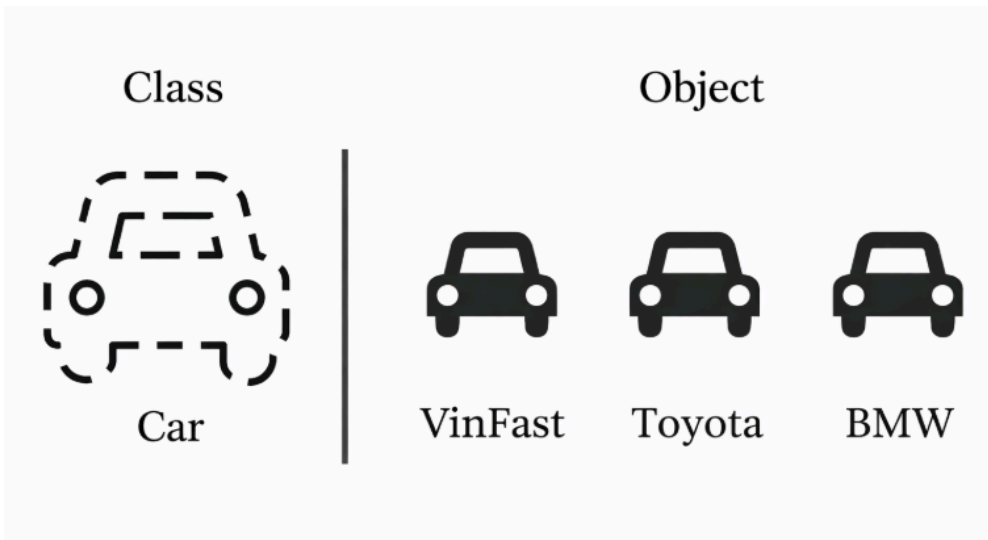


Figure 5-1. Objects Created from a Template: the Car Class

Object-oriented and procedural programming have many differences:

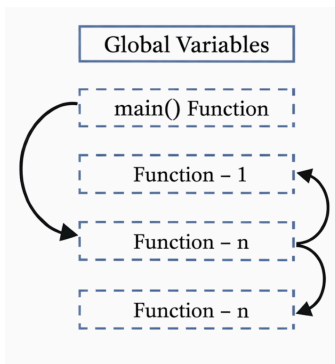


Figure 5-2. Procedure-oriented programming

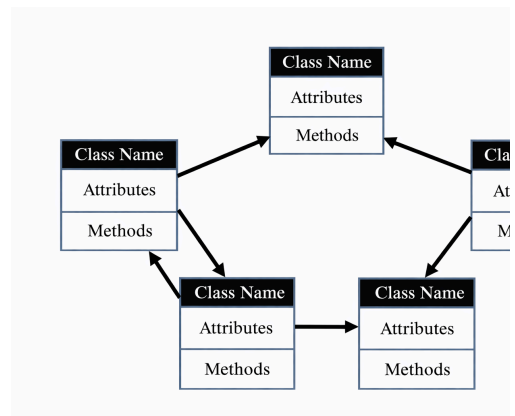


Figure 5-3. Object-Oriented Programming

The advantage of object-oriented programming (OOP) over procedure-oriented programming is that it simplifies design, development, and maintenance. In contrast, procedure-oriented programming makes source code management difficult, especially for large projects. As a result, OOP improves the efficiency of the software development process. Additionally, OOP enables the modeling of real-world entities and

behaviors, thereby making software development more intuitive and effective.

Criteria	Procedural Programming	Object-Oriented Programming
Programming Paradigm	Procedural programming	Object-oriented programming
Approach	Top-down	Bottom-up
Program Decomposition	Program is divided into functions	Program is divided into classes and objects
Design Structure	Transform the program into functions	Divide the program into layers
Access Control	No access specification is provided	Access is defined using <i>public</i> , <i>private</i> , <i>protected</i> , and <i>default</i>
Method Overloading and Polymorphism	Not supported	Supported through method overloading and polymorphism
Inheritance	Inheritance is not supported	Inheritance is supported using <i>public</i> , <i>protected</i> , and <i>private</i>
Data Sharing	Data is shared among functions	Data is shared among objects if permitted
Commonly Used Languages	TC, VB, Pascal	C++, Java, C#, Ruby

5.1.2. Characteristics of Object-Oriented Programming



Figure 5-4. Illustration of objects in a video game.

- **Inheritance:** Inheritance occurs when an object acquires the properties and behaviors of its parent object. This mechanism promotes code reusability.
- **Polymorphism:** Polymorphism refers to the ability of an object to take multiple forms, allowing it to exhibit different behaviors under different conditions.
- **Abstraction:** Abstraction is the process of hiding unnecessary implementation details and exposing only the essential features of an object, focusing on what an object does rather than how it does it.
- **Encapsulation:** Encapsulation is the practice of binding data and methods into a single unit (a class, a package) and restricting direct access to them. This helps protect the internal state of an object from external interference.

5.1.3. Some concepts about classes, objects, attributes, and methods

- A **class** defines the structure and behavior of objects. It can also be

considered a user-defined data type whose instances are treated as objects.

- An **object** is an instance of a class.
- An **attribute** is a property or characteristic that describes an object.
- A **method** represents the behavior or actions that an object can perform.

Consider the following program segment, which defines a **Student** class using the **class** keyword:

```
class Student {
    String name;
    int age;

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void display() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }

    public static void main(String[] args) {
        Student s1 = new Student("Linh", 24);
        s1.display();
    }
}
```

Result:

Name: Linh

Age: 24

The syntax for declaring a class is as follows:

```
class <class name> {
    // Declare a list of attributes
```

```

    <data_type> <attribute_name>;
    ...
    // Declare a list of methods
    public <return type> <method name>(<parameter type>
<parameter name>, ...) {
        ...
    }
}

```

Create a Student class with two attributes, name and age, and two methods, `getInformation()` and `display()`, as shown in the following table:

Student
~ name:String ~ age: int
+display(): void +getInformation(): void

Figure 5-5. Declaring attributes and methods for the Student class.

```

class Student {
    String name;
    int age;

    public void getInformation() {
    }

    public void display() {
    }
}

```

Use the keyword `'new'` to declare an object of a class. To declare an object of the **Student** class :

```

Student s1 = new Student( "Linh" , 24 );

```

```
s1.display ();
```

Use the **"."** **operator** to access the properties and methods of an object.

```
public class Program {  
    public static void main(String[] args) {  
        Circle c = new Circle(7);  
        System.out.println(c.getArea());  
        System.out.println(c.getCircumference());  
    }  
}  
  
class Circle {  
    private double radius;  
  
    public Circle(double radius) {  
        this.radius = radius;  
    }  
  
    public double getArea() {  
        return 3.14 * radius * radius;  
    }  
  
    public double getCircumference() {  
        return 3.14 * 2 * radius;  
    }  
}
```

Result:

```
153.86
```

```
43.96
```

5.2. COMPARING TWO OBJECTS

Consider the Student object:

```
public class Student {
```

```
public String name;
public String address;

public Student(String name, String address) {
    this.name = name;
    this.address = address;
}
}
```

The Program class with the **main** method :

```
public class Program {
    public static void main(String[] args) {
        Student s1 = new Student("Viet", "Bac Ninh");
        Student s2 = new Student("Viet", "Bac Ninh");
        System.out.println(s1 == s2);
    }
}
```

The results of the run are:

```
false
```

Although the two objects **s1** and **s2** are created with identical values for **name** and **address**, the expression **s1 == s2** returns **false**. This is because the **==** operator in Java compares **object references**, that is, the memory addresses where the objects are allocated, rather than the values of their attributes.

To compare object values, you must define a custom comparison method or override the **equals()** method inherited from the **Object** class, as shown in the following program:

```
public class Student {
    public String name ;
    public String address ;

    public Student ( String name, String address) {
```



```

        this.name = name ;
        this.address = address ;
    }

    @Override
    public boolean equals ( Object obj) {
        return this . name == (( Student ) obj). name && this .
address == (( Student ) obj). address ;
    }
}

```

The program class with the main method:

```

public class Program {
    public static void main ( String [] args) {
        Student s1 = new Student( "Viet" , "Bac Ninh" );
        Student s2 = new Student( "Viet" , "Bac Ninh" );
        System . out .println( s1 .equals( s2 ));
    }
}

```

Result:

True

5.3. ACCESS MODIFIERS

Access modifiers are **keywords** used before the declaration of a class, variable, or method to specify the level of access permitted to other classes.

For **classes**, Java provides two access levels: **public** and **default**.

For **variables and methods**, Java provides four access modifiers: **public**, **protected**, **default**, and **private**.

- **default** (no access modifier specified): Accessible within the same package.
- **private**: Accessible only within the same class.
- **public**: Accessible from anywhere.

- **protected**: Accessible within the same package or by subclasses outside the package.

Summary table:

Table 5-1. Summary table of access modifiers

Access Modifier	Same Class	Same Package	Subclasses Outside the Package	Other Classes Outside the Package
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

5.4. GETTER AND SETTER

If a class attribute is declared as **public**, it can be accessed directly from outside the class. To reduce the use of public attributes and to protect the internal state of an object, attributes should be declared as **private**. Access to these attributes is then provided through public methods, commonly known as **getter** and **setter** methods.

- **Getter** methods provide controlled read access to private attributes.
- **Setter** methods provide controlled write access to private attributes.

5.4.1. Getter

Getter methods are methods that provide read access to a variable.

```
public class Student {
    private String name ;
```

```
private integer age ;  
private double gpa ;  
}
```

The following example provides read access to the variable **name**:

```
public String getName () {  
    return name ;  
}
```

By convention, the name of a getter method follows the form **get + variable name**, with the first letter of the variable name capitalized. For example: **getName()**.

5.4.2. Setter

Setter methods provide "write access" to a private variable, allowing other classes to assign a new value to the variable.

```
public void setName ( String name) {  
    this.name = name ;  
}
```

By convention, the name of a setter method begins with **set** followed by the capitalized variable name, and it must accept a parameter of the same type as the variable.

5.5. CONSTRUCTOR METHOD

5.5.1. Concept of Constructors Methods

A **constructor** is a special method that is automatically invoked when an object is created. Its purpose is to initialize the newly created object.

Constructors have the following characteristics:

- A constructor has the **same name as the class**.
- A constructor **does not declare a return type**, not even **void**.

Unlike regular methods, which are invoked using the dot (.) operator, a constructor is invoked using the **new** operator.

Objects are created using the **new** operator. For example, the following program creates an object of the **Student** class:

```
Student s1 ; //Declare a variable of type Student
s1 = new Student(); // Creates a new object and stores a
reference to it.
```

In this example, the expression **new Student()**:

- Allocates memory for the object,
- Initializes the object's instance variables,
- Returns a reference to the newly created object.

After the assignment statement is executed, **s1** refers to the newly created object. The expression **Student()** resembles a method call; in fact, it is a call to a special method known as a **constructor**.

Every Java class has at least one constructor. If a programmer does not explicitly define a constructor, Java automatically provides a **default constructor** for that class.

Rules for Creating Constructors

- The constructor name must be **identical to the class name**.
- A constructor **has no return type**.
- A constructor cannot be declared as **abstract**, **static**, **final**, or **synchronized**.
- A constructor may be declared with access modifiers such as

`public`, `protected`, or `private`.

- A constructor **cannot be static**.

In Java, constructors are classified into two types:

- **Parameterless (no-argument) constructors**
- **Parameterized constructors**

5.5.2. Parameterless constructor

A **parameterless constructor** (also called a **no-argument constructor**) is a constructor that does not take any parameters. It is used to create an object with default initial values.

Example of a parameterless constructor in the `Student` class:

```
public class Student {  
    String name;  
    int age;  
  
    public Student() {  
    }  
}
```

Key points about parameterless constructors:

- A parameterless constructor **can be explicitly defined** by the programmer, as shown above.
- If a class **does not define any constructor**, Java automatically provides a **default parameterless constructor**.
- The automatically generated constructor initializes instance variables to their default values (e.g., `null` for objects, `0` for numeric types,

false for boolean).

Once a constructor with parameters is explicitly defined, the system **no longer generates** the default parameterless constructor unless it is explicitly declared.

5.5.3. Parameterized constructors

A **parameterized constructor** is a constructor that allows values to be passed to an object at the time of its creation. These values are used to initialize the instance variables of the class.

Example:

```
public class Student {  
    String name;  
    int age;  
  
    public Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

Key points to note:

- A parameterized constructor enables **initialization of instance variables with specific values** at the time of object creation.
- Once a constructor is explicitly declared in a class, the **default parameterless constructor is no longer automatically generated** by Java.
- If a parameterless constructor is still required, it **must be explicitly defined** in the class.

5.6. OBJECT ARRAY

Declaring and using arrays of objects differs from declaring and using arrays of primitive data types such as `int` or `double`. When working with an array of objects, each element of the array must be **explicitly initialized** before it can be used.

For example, suppose `Student` is a user-defined class. An array of type `Student[]` contains references to `Student` objects. To store information for 30 students, an array can be declared and initialized as follows:

```
public static void main ( String [] args) {
    // Declare an array of students with 30 elements
    Student [] students = new Student[ 30 ];
    for ( int i = 0 ; i < 30 ; i ++ ) {
        // Initialize the elements in the students array
        students [ i ] = new Student();
    }
}
```

This array contains 30 elements: `students[0]`, `students[1]`, ..., `students[29]`. After initialization, each `students[i]` references a `Student` object.

Consider the following definition of the `Student` class:

```
public class Student {
    String name;
    int age;

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void display() {
        System.out.println("Name: " + name);
    }
}
```

```
        System.out.println("Age: " + age);
    }
}
```

The **Program** class demonstrates how to use an array of **Student** objects:

```
public class Program {
    public static void main(String[] args) {
        // Declare an array of students with 3 elements
        Student[] students = new Student[3];

        students[0] = new Student("Thai", 12);
        students[1] = new Student("Tuan", 13);
        students[2] = new Student("Duc", 11);

        for (int i = 0; i < 3; i++) {
            students[i].display();
        }
    }
}
```

The output of the program is:

```
Name: Thai
Age: 12
Name: Tuan
Age: 13
Name: Duc
Age: 11
```

5.7. SOME COMMON CLASSES

5.7.1. The `java.lang.StringBuilder` class

The **StringBuilder** class, which belongs to the standard **java.lang** package, is used to efficiently build and modify strings composed of

multiple parts. Unlike the `String` class, which is immutable, `StringBuilder` allows its content to be changed without creating new objects.

To use `StringBuilder`, you first create an instance of the class:

```
StringBuilder builder = new StringBuilder();
```

Similar to a `String`, a `StringBuilder` maintains a sequence of characters. A long string can be constructed by repeatedly appending smaller strings using the `append()` method. Once the construction is complete, the `toString()` method is used to obtain the final result as a `String` object.

The following example compares the execution time of string concatenation using the `String` class and the `StringBuilder` class:

```
public class Program {
    private static final String STR = "A";

    public static String stringConcatenation() {
        String s = STR;
        for (int i = 0; i < 100000; i++)
            s = s + STR;
        return s;
    }

    public static String stringBuilderConcatenation() {
        StringBuilder sb = new StringBuilder(STR);
        for (int i = 0; i < 100000; i++)
            sb.append(STR);
        return sb.toString();
    }

    public static void main(String[] args) {
```

```

        long start = System.currentTimeMillis();
        stringConcatenation();
        long end = System.currentTimeMillis();
        System.out.println("Time taken to concatenate String: "
+ (end - start) + " ms");

        start = System.currentTimeMillis();
        stringBuilderConcatenation();
        end = System.currentTimeMillis();
        System.out.println("Time taken to concatenate strings
using StringBuilder: " + (end - start) + " ms");
    }
}

```

Result:

Time taken to concatenate String: 3542 ms

Time taken to concatenate strings using StringBuilder: 2 ms

5.7.2. The java.util.Date class

The `java.util.Date` class is used to represent a specific point in time, including both date and time information. When a `Date` object is created without any parameters, it represents the **current date and time**.

```

public static void main ( String [] args) {
    System . out .println( new Date());
}

```

The output is displayed in a readable date–time format, for example:

Sun Sep 12 07:58:44 ICT 2021

This output reflects the current date and time according to the system's default time zone.

5.7.3. The java.lang.Math class

The `Math` class provides a collection of **static methods** for performing common mathematical operations, such as trigonometric functions (`sin`,

`cos`, `tan`), rounding methods, and random number generation.

For example, the `random()` method returns a pseudorandom value of type `double` in the range `[0.0,1.0)[0.0, 1.0)[0.0,1.0)`:

```
double x = Math.random();
```

The following example defines a `PairOfDice` class. Each object of this class represents a pair of dice, using two instance variables to store the values shown on the dice. When a `PairOfDice` object is created, the dice are initialized with random values.

```
public class PairOfDice {
    public int dice1 , dice2 ;
    public PairOfDice () {
        roll();
    }

    public void roll () {
        dice1 = ( int ) ( Math . random () * 6 ) + 1 ;
        dice2 = ( int ) ( Math . random () * 6 ) + 1 ;
        System . out .println( "Dice 1 = " + dice1 );
        System . out .println( "Dice 2 = " + dice2 );
    }

    public static void main ( String [] args) {
        PairOfDice pairOfDice = new PairOfDice();
    }
}
```

A possible output of the program is:

```
Dice 1 = 5
Dice 2 = 6
```

Note that, in addition to the `Math.random()` method, Java also provides the `java.util.Random` class, which can be used to create a

Random object for generating random numbers with greater flexibility and control.

5.8. INHERITANCE

In object-oriented programming, **inheritance** is the mechanism by which one class acquires the attributes and methods of another class. A class that inherits attributes and methods from another class is called a **derived class** or **subclass**, while the class from which the attributes and methods are inherited is called a **base class** or **parent class**.

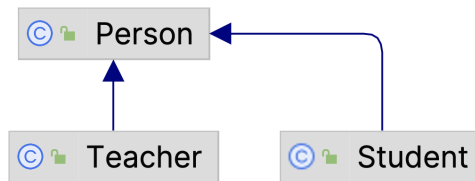


Table 5-2. The Teacher and Student classes both inherit from the Person class.

In Java, inheritance is implemented using the **extends** keyword:

```
class B extends A {
    // Additions and modifications
    // Content inherited from class A
}
```

For example:

```
public class Student extends Person {
}

public class Teacher extends Person {
}
```

The **Student** and **Teacher** classes inherit common attributes and methods from the **Person** class. In this relationship, **Person** is the **parent (base) class**, while **Student** and **Teacher** are **child (derived) classes**.

In Java inheritance:

- **Private** attributes and methods of a parent class are **not directly accessible** by subclasses.
- Subclasses can inherit attributes and methods declared as **public**, **protected**, or **default** (when the subclass and parent class are in the same package).
- Access to a parent class's private members must be provided through **getter and setter methods**.
- A class declared with the **final** keyword **cannot be inherited**.

5.9. OVERLOADING METHODS

In inheritance, a subclass can **override** a method of its parent class to provide a specific implementation. This is illustrated in the following program:

```
class SuperClass {
    public void display() {
        System.out.println("Hello from SuperClass");
    }
}

class SubClass extends SuperClass {
    @Override
    public void display() {
        System.out.println("Hello from SubClass");
    }
}

class Entry {
    public static void main(String[] args) {
        SubClass s = new SubClass();
        s.display();
    }
}
```

The output of the program is:

```
Hello from SubClass
```

In this example, the `display()` method of `SuperClass` is **overridden** by the `display()` method in `SubClass`. When the method is invoked on a `SubClass` object, the overridden version in the subclass is executed.

The `@Override` annotation indicates that the method is intended to override a method in the parent class. Although its use is not mandatory, it is strongly recommended for the following reasons:

- The compiler verifies that the annotated method correctly overrides a method in the parent class and reports an error if it does not.
- It improves code readability by clearly indicating that the method overrides a superclass method.

Note that a method declared with the `final` keyword **cannot be overridden** by subclasses.

5.10. THE JAVA.LANG.OBJECT CLASS

Every class in Java implicitly inherits from the `Object` class. If a class is declared without explicitly extending another class, it automatically becomes a subclass of the special `Object` class, which is defined in the `java.lang` package.

The `Object` class provides several methods that are inherited by all Java classes.

Methods of the `Object` Class:

Method	Description
--------	-------------

<code>public final Class<?> getClass()</code>	Returns the runtime class of the object.
<code>public int hashCode()</code>	Returns a hash code value for the object.
<code>public boolean equals(Object obj)</code>	Compares this object with another object for equality.
<code>protected Object clone() throws CloneNotSupportedException</code>	Creates and returns a copy of the object.
<code>public String toString()</code>	Returns a string representation of the object.
<code>public final void notify()</code>	Wakes up a single thread waiting on this object.
<code>public final void notifyAll()</code>	Wakes up all threads waiting on this object.
<code>public final void wait() throws InterruptedException</code>	Causes the current thread to wait until it is notified.

`public final void wait(long timeout)` throws `InterruptedException` Causes the current thread to wait for a specified time.

`public final void wait(long timeout, int nanos)` throws `InterruptedException` Causes the current thread to wait for a specified time with nanosecond precision.

`protected void finalize()` throws `Throwable` Called by the garbage collector before the object is reclaimed.

A class can **override** methods inherited from the `Object` class to provide behavior specific to that class. A commonly overridden method is `toString()`.

The following example overrides the `toString()` method in the `PairOfDice` class introduced earlier:

```
public class PairOfDice {
    public int dice1, dice2;

    public PairOfDice() {
        roll();
    }

    @Override
    public String toString() {
        if (dice1 == dice2)
            return "Double " + dice1;
        else
            return dice1 + " and " + dice2;
    }
}
```



```
}

public void roll() {
    dice1 = (int) (Math.random() * 6) + 1;
    dice2 = (int) (Math.random() * 6) + 1;
}

public static void main(String[] args) {
    PairOfDice pairOfDice = new PairOfDice();
    System.out.println(pairOfDice.toString());
}
}
```

In this example, the overridden `toString()` method replaces the inherited version from the `Object` class, providing a more meaningful string representation of a `PairOfDice` object.

5.11. THIS AND SUPER

5.11.1. The keyword "this"

The keyword `this` is an important feature in Java that refers to the **current object**. It is commonly used to access the object's instance variables and methods.

The `this` keyword is especially useful for distinguishing between **instance variables** and **method parameters** when they share the same name. This situation frequently occurs in constructor methods.

Consider the following example:

```
public class Circle {
    private double radius;

    public Circle(double radius) {
        radius = radius;
    }
}
```

```
}

public static void main(String[] args) {
    Circle circle = new Circle(3.12);
    System.out.println("radius = " + circle.radius);
}
}
```

The output of the program is:

```
radius = 0
```

In this example, the constructor parameter `radius` has the same name as the instance variable `radius`. Inside the constructor, the assignment `radius = radius` assigns the parameter to itself rather than to the instance variable. As a result, the instance variable remains unchanged and retains its default value (`0.0` for type `double`).

To resolve this ambiguity, the `this` keyword is used to explicitly refer to the instance variable of the current object.

The corrected program is shown below:

```
public class Circle {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    public static void main(String[] args) {
        Circle circle = new Circle(3.12);
        System.out.println("radius = " + circle.radius);
    }
}
```

Run results:

```
radius = 3.12
```

By using `this.radius`, the constructor correctly assigns the parameter value to the instance variable of the object.

5.11.2. The keyword "super"

The keyword `super` is used to refer to members of a **parent (superclass)** from within a **child (subclass)**. It is commonly used to resolve name conflicts when a subclass overrides a method defined in its parent class.

When a subclass defines a method with the same name as a method in its superclass, the subclass method **overrides** the superclass method. However, the `super` keyword can be used to explicitly invoke the overridden method of the parent class.

Consider the following example:

```
class Animal { // Superclass (parent)
    public void animalSound() {
        System.out.println("The animal makes a sound");
    }
}

class Dog extends Animal { // Subclass (child)
    @Override
    public void animalSound() {
        super.animalSound();
        System.out.println("The dog says: bow wow");
    }
}
```

Main program:

```
public static void main(String[] args) {
    Animal myDog = new Dog();
    myDog.animalSound();
}
```

The output of the program is:

```
The animal makes a sound
```

The dog says: bow wow

In this example, the `animalSound()` method in the `Dog` class overrides the method in the `Animal` class. The call to `super.animalSound()` ensures that the parent class's version of the method is executed before the subclass's additional behavior.

5.14. ABSTRACT

Abstraction is a fundamental concept in object-oriented programming that focuses on exposing only the **essential features** of an object while hiding unnecessary implementation details. In Java, abstraction is achieved through **abstract classes** and **interfaces**.

A class declared with the `abstract` keyword **cannot be instantiated**. It is intended to be used only as a superclass for other classes.

Example:

```
abstract class Animal {  
}  
class Cat extends Animal {  
}
```

The statement `Animal a = new Cat();` is valid, because a reference of an abstract class type can refer to an object of a subclass. However, attempting to create an object using `new Animal()` will result in a compilation error, since abstract classes cannot be instantiated.

An abstract class may contain **abstract methods**. An abstract method is declared without an implementation body.

Example:

```
abstract class Animal {  
    public abstract void makeSound ();  
}
```

```
}
```

If a class inherits from an abstract class, it **must override all abstract methods**, unless the subclass itself is declared abstract.

Example:

```
class Cat extends Animal {  
    @Override  
    public void makeSound () {  
        System . out .println( "meow meow" );  
    }  
}
```

A **Cat** object can then be created, and the overridden method can be invoked as follows:

```
public class Program {  
    public static void main ( String [] args) {  
        Animal a = new Cat();  
        a.makeSound ();  
    }  
}
```

The output of the program is:

```
meow meow
```

This example demonstrates how abstraction allows programs to work with general types while relying on specific implementations provided by subclasses.

5.12. INTERFACE

An interface is used to define a set of abstract behaviors and constants that a class can implement. Interfaces support abstraction and help achieve multiple inheritance of behavior in Java.

Key characteristics of interfaces include:

- An interface is declared using the **interface** keyword.
- Interfaces **cannot be instantiated**.
Interfaces are implemented by classes or extended by other interfaces.
- All methods in an interface are **abstract by default**, except for **default** and **static** methods.
- All variables declared in an interface are implicitly **public**, **static**, and **final** (constants).
- The **abstract** keyword is not required when declaring abstract methods.
- The **static** and **final** keywords are not required when declaring constants.

For example, the **IAAnimal** interface can be defined as follows:

```
public interface IAnimal {  
    default void move () {  
        System . out .println( "Moving" );  
    }  
    void sound ();  
    int numberOfAnimal = 0;  
}
```

A class can inherit from multiple interfaces. Because Java is a language designed for simplicity, it doesn't support multiple inheritance with classes, but due to the nature of interfaces containing only abstract (empty) methods, Java allows a class to implement multiple interfaces.

```
interface IFlyable {  
    void fly ();  
}  
  
interface IEatable {  
    void eat ();  
}
```

```

}

class Bird implements IFlyable , IEatable {
    @Override
    public void fly () {
        System . out .println( "Bird flying" );
    }

    @Override
    public void eat () {
        System . out .println( "Bird eats" );
    }
}

```

Main program:

```

public class Program {
    public static void main ( String [] args) {
        Bird bird = new Bird();
        bird.eat ();
        bird.fly ();
    }
}

```

The result when running the program:

```

Bird eats
Bird flying

```

Note : To implement an interface, use the keyword **`implements`** instead of **`extends`** . When looking at an interface, the only thing visible are the abstract methods (features); therefore, using an interface is considered completely abstract. Interfaces are usually named with a capital I at the beginning, often followed by a noun or adjective.

5.13. POLYMORPHISM

Polymorphism refers to the ability of an object to take **multiple forms** and to exhibit different behaviors depending on the actual object type at runtime.

In Java, polymorphism is primarily achieved through **method overriding** and **dynamic method binding**.

Consider the following example.

```
class Animal {
    public void sound() {
        System.out.println("some sound");
    }
}

// The Dog class inherits from Animal
class Dog extends Animal {
    @Override
    public void sound() {
        System.out.println("bow bow");
    }
}

// The Cat class inherits from Animal
class Cat extends Animal {
    @Override
    public void sound() {
        System.out.println("meow meow");
    }
}

// The Duck class inherits from Animal
class Duck extends Animal {
    @Override
    public void sound() {
        System.out.println("quack quack");
    }
}
```

Main program:


```
public class Program {  
    public static void main(String[] args) {  
        Animal animal = new Animal();  
        animal.sound();  
  
        animal = new Dog();  
        animal.sound();  
  
        animal = new Cat();  
        animal.sound();  
  
        animal = new Duck();  
        animal.sound();  
    }  
}
```

The output of the program is:

```
some game  
bow bow  
meow meow  
quack quack
```

In this example:

- The reference variable `animal` is of type `Animal`.
- At runtime, it can refer to objects of different subclasses such as `Dog`, `Cat`, and `Duck`.
- Although the reference type remains the same, the method implementation that is executed depends on the **actual object type**.

This behavior demonstrates **runtime polymorphism**, where the same method call (`sound()`) results in different behaviors based on the object

being referenced.

To determine whether an object belongs to a specific class or subclass, Java provides the **instanceof** keyword.

```
public class Program {  
    public static void main(String[] args) {  
        Animal animal = new Dog();  
  
        if (animal instanceof Dog) {  
            System.out.println("animal is an instance of the  
Dog class");  
        }  
    }  
}
```

Key points about **instanceof**:

- It returns **true** if the object is an instance of the specified class or its subclasses.
- It is commonly used when type-specific behavior is required.
- Excessive use of **instanceof** may indicate poor design; polymorphism is generally preferred over explicit type checking.

Polymorphism improves **code flexibility**, **extensibility**, and **maintainability**, allowing programs to be written in a more general and reusable manner.

PRACTICE

Lesson 1. Inheritance

Create a Person class:

```
public class Person {  
    private int id ;  
    private String name ;  
    private integer age ;  
    private String address ;  
  
    public Person ( int id, String name, int age, String  
address) {  
        this.id = id ;  
        this.name = name ;  
        this.age = age ;  
        this.address = address ;  
    }  
  
    public int getId () {  
        return id ;  
    }  
  
    public void setId ( int id) {  
        this.id = id ;  
    }  
  
    public String getName () {  
        return name ;  
    }  
  
    public void setName ( String name) {  
        this.name = name ;  
    }  
  
    public int getAge () {  
        return age ;  
    }  
  
    public void setAge ( int age) {  
        this.age = age ;  
    }  
}
```

```
}

    public String getAddress () {
        return address ;
    }

    public void setAddress ( String address) {
        this.address = address ;
    }
}
```

Create a Student class that inherits from the Person class:

```
public class Student extends Person {
    int gpa ;

    public Student ( int id, String name, int age, String
address, int gpa) {
        super (id, name, age, address);
        this.gpa = gpa ;
    }

    public int getGpa () {
        return gpa ;
    }

    public void setGpa ( int gpa) {
        this.gpa = gpa ;
    }
}
```

Create a Teacher class that inherits from the Person class:

```
public class Teacher extends Person {
    Private income salary ;

    public Teacher ( int id, String name, int age, String
address, int salary) {
        super (id, name, age, address);
        this.salary = salary ;
    }

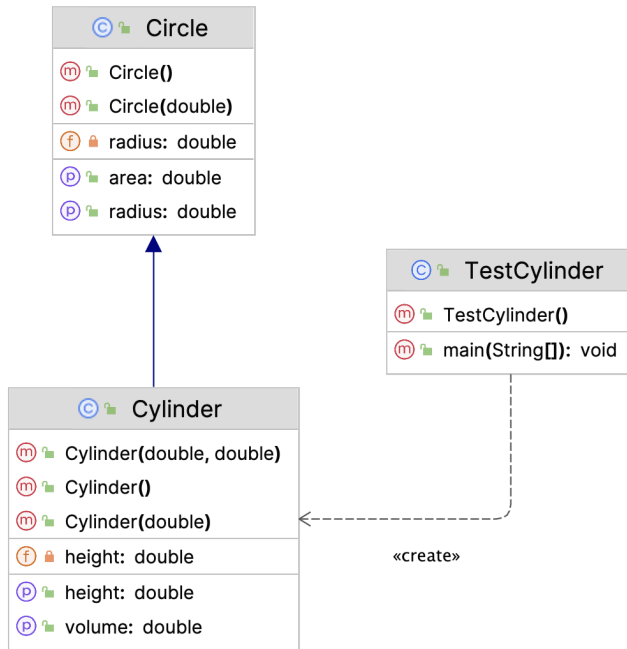
    public int getSalary () {
        return salary ;
    }

    public void setSalary ( int salary) {
        this.salary = salary ;
    }
}
```

```
}
```

Lesson 2. Inheritance

Create a base class named Circle and another class named Cylinder that inherits from Circle. Then, write a program to use these classes. The program diagram is as follows:



Create a Circle class as follows:

```
public class Circle {
    private double radius ;
    private String color ;

    public Circle() {
        radius = 1.0 ;
        color = "red" ;
    }

    public Circle( double r) {
        radius = r;
```

```

        color = "red" ;
    }

    public double getRadius() {
        return radius ;
    }

    public double getArea() {
        return radius * radius * Math.PI ;
    }
}

```

Create a Cylinder class that inherits from the Circle class:

```

public class Cylinder extends Circle {
    private double height ;

    public Cylinder() {
        super () ;
        height = 1.0 ;
    }

    public Cylinder( double height) {
        super (); // call superclass no-arg constructor
        this.height = height ;
    }

    public Cylinder( double radius, double height) {
        super (radius); // call superclass constructor
        this.height = height ;
    }

    public double getHeight() {
        return height ;
    }
}

```

```
public double getVolume() {  
    return getArea() * height ;  
}  
}
```

Create a TestCylinder class to test the classes you have created:

```
public class TestCylinder { //save as "TestCylinder.java"  
    public static void main(String[] args) {  
        Cylinder c1 = new Cylinder();  
        System.out.println( "Cylinder:"  
            + " radius=" + c1.getRadius()  
            + " height=" + c1.getHeight()  
            + " base area=" + c1.getArea()  
            + " volume=" + c1.getVolume());  
        Cylinder c2 = new Cylinder( 10.0 );  
        System.out.println( "Cylinder:"  
            + " radius=" + c2.getRadius()  
            + " height=" + c2.getHeight()  
            + " base area=" + c2.getArea()  
            + " volume=" + c2.getVolume());  
        Cylinder c3 = new Cylinder( 2.0 , 10.0 );  
        System.out.println( "Cylinder:"  
            + " radius=" + c3.getRadius()  
            + " height=" + c3.getHeight()  
            + " base area=" + c3.getArea()  
            + " volume=" + c3.getVolume());  
    }  
}
```

Execute the program. The result when running the program is as follows:

```
Cylinder: radius=1.0 height=1.0 base area=3.141592653589793  
volume=3.141592653589793
```

```
Cylinder: radius=1.0 height=10.0 base area=3.141592653589793
```



```
volume=31.41592653589793
```

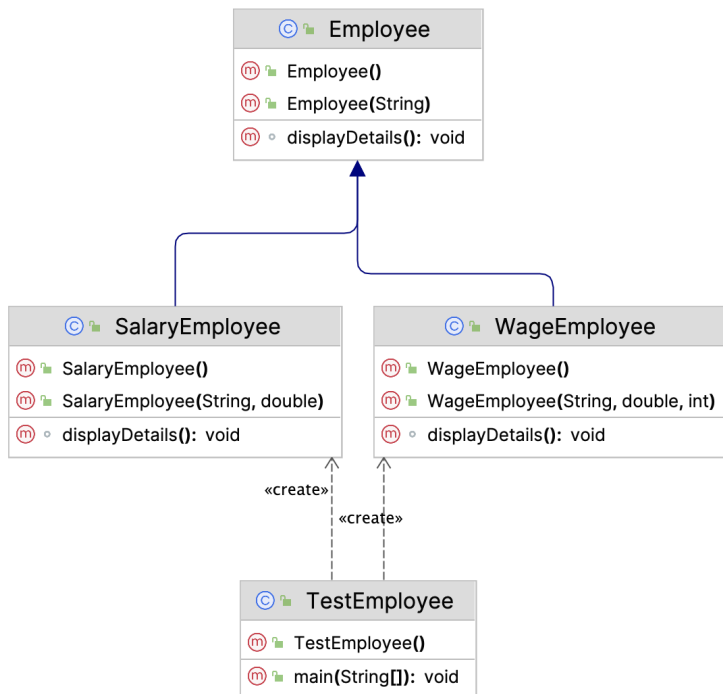
```
Cylinder: radius=2.0 height=10.0 base area=12.566370614359172  
volume=125.66370614359172
```

Lesson 3. Practice

Create a program to work with two types of employees:

- **Permanent staff** : receive a fixed salary.
- **Contract employees** : have a fixed salary and working hours.

The program's class diagram is as follows:



Create the Employee class by following these steps:

```
public class Employee {  
  
}
```

Declare a variable to store employee names:

```
String empName ;
```

Create two constructors:

```
public Employee() {  
}  
  
public Employee(String name) {  
    empName = name;  
}
```

Create a method to display the name:

```
void displayDetails() {  
    System.out.printf( "Employee Name: %s" , empName );  
}
```

Create a SalaryEmployee class that inherits from the Employee class:

```
public class SalaryEmployee extends Employee {  
}
```

Declare a variable called `salary` to store employee salaries:

```
double salary ;
```

Declare constructor methods:

```
public SalaryEmployee() {  
}  
  
public SalaryEmployee(String name, double currentSalary) {  
    super (name);  
    salary = currentSalary;  
}
```

Methods to display employee salary details:

```
void displayDetails() {  
    super.displayDetails ();  
    System.out.printf( " \n " + empName + "'s Salary: %.2f \n "  
        , salary );  
}
```

WageEmployee inherits from the Employee class:

```
public class WageEmployee extends Employee {  
}
```

Declare additional attributes for the WageEmployee class:

```
double rate ;  
int hours ;
```

Declare two additional constructors:

```
public WageEmployee() {  
}  
  
public WageEmployee(String name, double wageRate, int  
wageHours) {  
    super (name);  
    rate = wageRate;  
    hours = wageHours;  
}
```

Create a method to display employee salary details:

```
void displayDetails() {  
    super.displayDetails ();  
    System.out.printf( " \n " + empName + "'s Salary: %.2f \n "  
    , rate * hours );  
}
```

Create a TestEmployee class with a main method to execute the main program:

```
public class TestEmployee {  
    public static void main(String[] args) {  
        Employee objJohn = new SalaryEmployee( "John" , 2300.50 );  
        objJohn.displayDetails();  
        Employee objDavid = new WageEmployee( "David" , 34.50 , 10 );  
        objDavid.displayDetails();  
    }  
}
```

Execute the program and print the output:

```
Employee Name: John
John's Salary: 2300.50
Employee Name: David
David's Salary: 345.00
```

QUIZ

The keyword `super` is used for:	
Reference to the interface	
Reference to a subclass	
Reference to the parent class	
Reference to a superobject	

To check if an object is an instance of a class, use:	
instanceof	
typeof	
isclass	
classof	

The keyword abstract is used to:	
Abstracting a block of code	
Create abstract classes or methods.	
Preventing inheritance	
Do not allow access to the property.	

To have one class inherit from another, use the keyword:	
Extend	
implements	
abstract	
inherit	

To prevent a property from being accessed outside the class, use the keyword:	
private	
master	
lockaccess	
By default	

The protected access setting is used for:	
Protect a property from change.	
Allow the inheriting class to access	
Prevent access by classes within the same package.	
Prevent access by subclasses.	

Getter and setter methods are used for:	
Get and set the value for the object.	
Set access permissions	
Prevent access to variables	
Customize property access permissions.	

Which statement about initialization methods is incorrect?	
The constructor has the same name as the class.	
The constructor method does not allow inheritance.	
A constructor must always be accompanied by a destructor.	
A class can have multiple constructors.	

Polymorphism is characterized by:	
An object can only have one unique behavior.	
An object can inherit from multiple sources.	
An object can take many different forms.	
One object can be related to many other objects.	

What output will be displayed when the following program is run ?	
<code>class Person {</code>	

```

private String name ;

    public String getName () {
        if ( name .length()> 16 )
            return "Name is too long!";
        Elle
return name ;
    }

    public void setName ( String name) {
        if (name!= null && name.length()> 2 )
            this.name = name ;
    }
}

public class Program {
public static void main ( String [] args) {
    Person person = new Person();
    person.setName ( "Nguyen Van Thanh" );
    System . out .println( person .getName());
}
}

```

An exception occurred.

A compilation error occurred.

The screen displays "Name is too long".

Nguyen Van Thanh's display screen

What happens when you compile/run the following code ?

```

interface MyInterface {
    int x = 1 ;
}

public class MyClass {
public static void main ( String [] args) {
    MyInterface . x = 2 ;
    System . out .println( MyInterface . x );
}
}

```

}	
Print the value 1.	
Print the value 2.	
Compilation error	
Error while running	

--

What is the output of the following code snippet ?	
<pre> class Parent { private String name ; private integer age ; public Parent () {} } class Child extends Parent { public Child () { name = "John" ; age = 42 ; } public static void main (String [] args) { Child child = new Child(); System . out .println(child); } } </pre>	
Compilation error	
John 42	
{name:"John", age:42}	
Child@xxxxxx where xxxxxx is a sequence of numbers	

What is the output of the following code snippet ?
--

<pre> public class Program { int field1, field2; String field3 ; public Program (int i, int j, String k) { field1 = i; field2 = j; field3 = k; } public static void main (String [] args) { Program foo1 = new Program(0 , 0 , "bar"); Program foo2 = new Program(0 , 0 , "bar"); System . out .println(foo1 .equals(foo2)); // prints false } } </pre>	
false	
true	
Compilation error	
The program makes an exception.	

What is the output of the following code snippet ? <pre> public interface MyInterface { void method1 (); int method2 (); String TEXT = "Hello" ; int ANSWER = 42 ; static void main (String [] args) { System . out .println(TEXT); } } </pre>	
Print the word "Hello"	
Compilation error	
The program makes an exception.	

EXERCISES

Lesson 1. Practice on Abstraction

The entire source code is written in a single class called Media, which includes the main method and the Item, MP3, and Book classes:

Create an Item class:

- It is an abstract class.
- It has a property containing a name (public) of type String.
- There is a description property (private) of type String that has a corresponding get set method.
- It has an attribute containing an ID (protected) of type String.
- It has a price attribute (default) of type float or int.
- There is an abstract method void showInfo().
- There is a constructor that takes the aforementioned attributes as parameters.

Create an MP3 layer:

- Inherit (extend) the Item class.
- There is an additional attribute called duration.
- There is a constructor with all the necessary attributes.

Create a Book class:

- Create a Book class that inherits from the Item class.
- There are additional properties: author (String), numberOfPages(int), genre (String).
- There is a constructor with all the necessary attributes.

Main program function

- Write a main() method that creates an MP3 object and a Book object.

- Call the `showInfo()` method of each object to display the object's attribute information.

Lesson 2. Practice on Polymorphism

Write code in the abstract class **Shape** , paying attention to completing the class by adding code to some methods.

```
public abstract class Shape {  
    private String color ;  
    private boolean protected ;  
  
    Shape() {  
  
    }  
  
    Shape(String color, boolean filled) {  
  
    }  
  
    String getColor() {  
  
    }  
  
    void setColor(String color) {  
  
    }  
  
    boolean isFilled() {  
  
    }  
  
    void setFilled( boolean filled) {  
  
    }  
  
    abstract double getArea();  
}
```

```
    abstract double getPerimeter();

    public abstract String toString();
}
```

Write code in the **Circle class** , paying attention to completing the class by **adding code** to some methods.

```
public class Circle extends Shape {
    private double radius ;

    Circle() {
    }

    Circle ( double radius) {

    }

    Circle( double radius, String color, boolean filled) {

    }

    public double getRadius() {

    }

    void setRadius( double radius) {

    }

    double getArea() {

    }

    double getPerimeter() {
```

```
}

    public String toString() {

}

}
```

Write code in the **Rectangle class** , paying attention to completing the class by **adding code** to some methods.

```
public class Rectangle extends Shape {
    private double width ;
    private double length ;

    Rectangle() {

    }

    @Override
    double getArea() {

    }

    @Override
    double getPerimeter() {

    }

    @Override
    public String toString() {

    }

    public double getWidth() {

    }

    public void setWidth( double width) {
```

```

}

    public double getLength() {

}

    public void setLength( double length) {

}

}

```

Write code in the **Square class** , paying attention to completing the class by **adding code** to some methods. Add code to the Square class.

```

public class Square extends Rectangle {

}

```

Use the code below to test all the classes created above . There are some errors in this code; explain them, fix them, and run the program.

```

public class TestAllShape {
    public static void main(String[] args) {
Shape s1 = new Circle( 5.5 , "RED" , false );
System.out.println ( s1);
System. out .println(s1.getArea());
System. out .println(s1.getPerimeter());
System. out .println(s1.getColor());
System. out .println(s1.isFilled());
System. out .println(s1.getRadius());

Circle c1 = (Circle)s1;
System.out.println ( c1);
System. out .println(c1.getArea());
System. out .println(c1.getPerimeter());
System. out .println(c1.getColor());
}
}

```

```
System.out.println(c1.isFilled());
System.out.println(c1.getRadius());

Shape s2 = new Shape();

Shape s3 = new Rectangle( 1.0 , 2.0 , "RED" , false ); //
Upcast
    System.out.println ( s3);
System.out.println(s3.getArea());
System.out.println(s3.getPerimeter());
System.out.println(s3.getColor());
System.out.println(s3.getLength());

Rectangle r1 = (Rectangle)s3; //downcast
    System.out.println ( r1);
System.out.println(r1.getArea());
System.out.println(r1.getColor());
System.out.println(r1.getLength());

Shape s4 = new Square( 6.6 ); // Upcast
    System.out.println ( s4);
System.out.println(s4.getArea());
System.out.println(s4.getColor());
System.out.println(s4.getSide());

Rectangle r2 = (Rectangle)s4;
System.out.println ( r2);
System.out.println(r2.getArea());
System.out.println(r2.getColor());
System.out.println(r2.getSide());
System.out.println(r2.getLength());

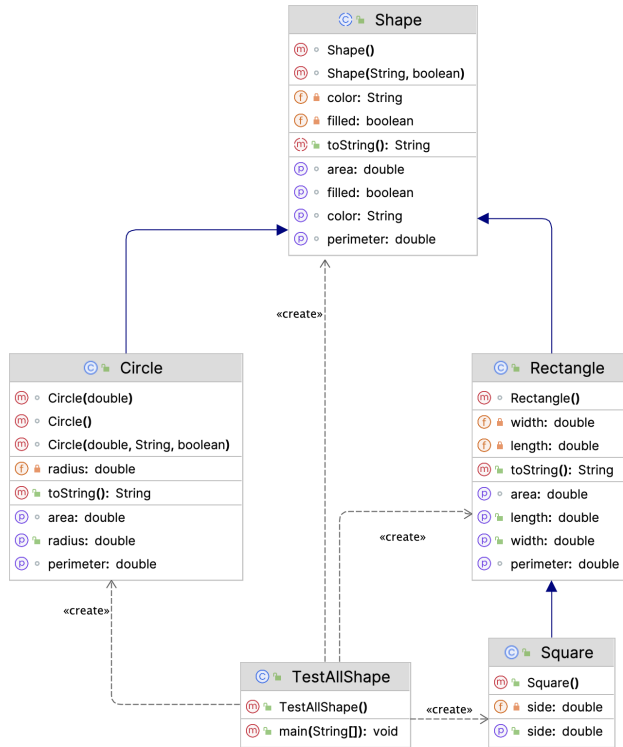
Square sq1 = (Square)s4;
System.out.println ( sq1);
System.out.println(sq1.getArea());
System.out.println(sq1.getColor());
```

```

System.out.println(sql.getSide());
System.out.println(sql.getLength());
}
}

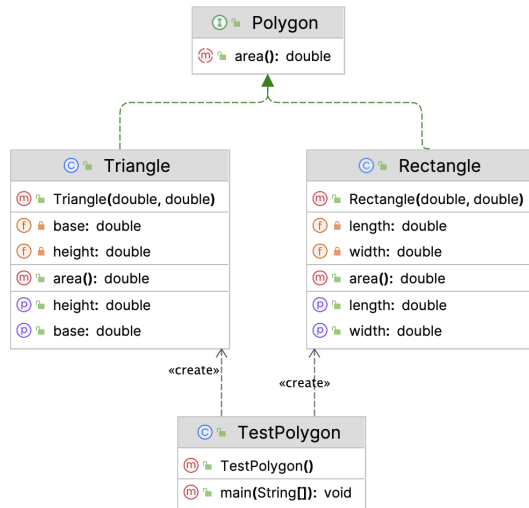
```

Please refer to the following class diagram:



Lesson 3. Practice on Interfaces

Design a Shape interface and define the basic behaviors that other classes will implement. The class diagram will look like this:



Lesson 4. Practical exercises on object-oriented programming in general.

Write code to create a Shape class with the following variables, constructor, and methods:

```
private String color ;

public Shape(String color) {
    this.color = color ;
}

public String toString() {
    return "Shape of color= \" " + color + " \" " ;
}

public double getArea() {
    System.err.println ( "I don't know what shape it is, so I
    can't calculate the area" );
    return 0 ; // Need a return to compile the program
}

```

Create a Rectangle class that inherits from the Shape class:

```
public class Rectangle extends Shape{

```



```
}
```

Declare additional properties for the Rectangle class:

```
private int length ;  
private int width ;
```

Write the constructor:

```
public Rectangle(String color, int length, int width) {  
    super (heat);  
    this.length = length ;  
    this.width = width ;  
}
```

Two methods override methods from the parent class:

```
@Override  
public String toString() {  
    return "Rectangle of length=" + length + " and width=" + width  
    + ", subclass of " + super.toString();  
}  
  
@Override  
public double getArea() {  
    return length * width ;  
}
```

Create a Triangle class that inherits from the Shape class:

```
public class Triangle extends Shape {  
}
```

Declare additional attributes for the Triangle class:

```
private int base ;  
private int height ;
```

Write the constructor:

```
public Triangle(String color, int base, int height) {  
    super (heat);  
    this.base = base ;  
    this.height = height ;  
}
```

```
}
```

Two methods override methods from the parent class:

```
@Override
public String toString() {
    return "Triangle of base=" + base + " and height=" + height
+ ", subclass of " + super.toString();
}

@Override
public double getArea() {
    return 0.5 * base * height ;
}
```

Create a TestShape class as the main program, and write the code for the main() method:

```
Shape s;

s = new Rectangle( "red" , 4 , 5 );
System.out.println ( s);
System.out.println( "Area is " + s.getArea());

s = new Triangle( "blue" , 4 , 5 );
System.out.println ( s);
System.out.println( "Area is " + s.getArea());

s = new Shape( "green" );
System.out.println ( s);
System.out.println( "Area is " + s.getArea());
```

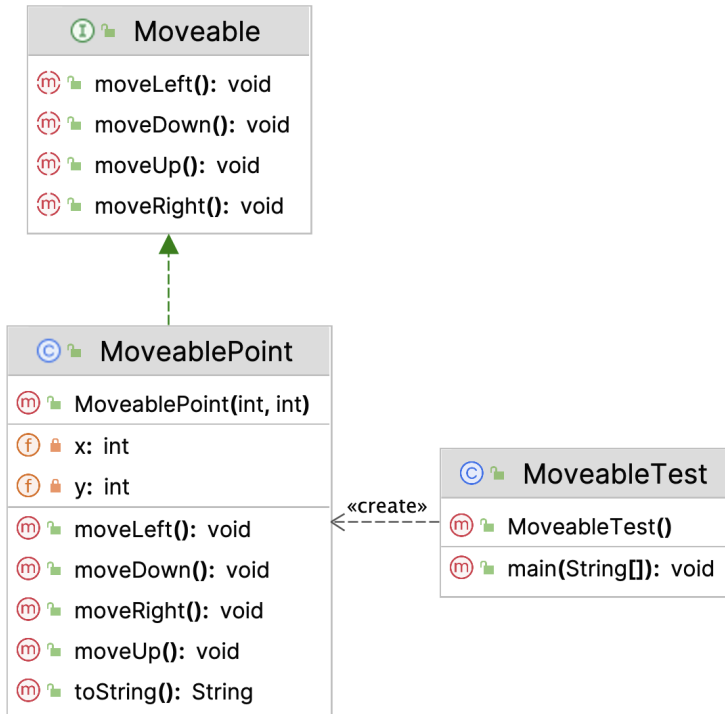
Run the main program and check the results:

```
Rectangle of length=4 and width=5, subclass of Shape of
color="red"
Area is 20.0
Triangle of base=4 and height=5, subclass of Shape of
color="blue"
Area is 10.0
```

```
Shape of color="green"
Area is 0.0
```

Lesson 5. Practice on Interfaces and Method Overloading

Given the class diagram of the program shown below:



Create an interface named **Moveable**:

```
interface Moveable {  
  
}
```

Declare abstract methods in the **Moveable** interface:

```
Void moveUp ();  
void moveDown ();  
void moveLeft ();  
Void moveRight ();
```

Create a **MoveablePoint** class that implements the **Moveable** Interface:

```
public class MoveablePoint implements Movable {

    @Override
    public void moveUp() {

    }

    @Override
    public void moveDown() {

    }

    @Override
    public void moveLeft() {

    }

    @Override
    public void moveRight() {

    }

}
```

Write a program for the MovablePoint class. Declare the variable (x, y) - the coordinates of the point:

```
private int x ;
private int y ;
```

Initialization method:

```
public MoveablePoint( int x, int y) {
    this.x = x ;
    this.y = y ;
}

public String toString() {
    return "Point at (" + x + ", " + y + ")" ;
}
```

```
}
```

Write code to implement the abstract methods defined by Movable:

```
@Override
public void moveUp() {
    y --;
}

@Override
public void moveDown() {
    y ++;
}

@Override
public void moveLeft() {
    x --;
}

@Override
public void moveRight() {
    x ++;
}
```

Create the main program class with the main method:

```
public static void main(String[] args) {
    Moveable m1 = new MoveablePoint( 5 , 5 );
    System.out.println ( m1);
    m1.moveDown();
    System.out.println ( m1);
    m1.moveRight();
    System.out.println ( m1);
}
```

Running the main program, the result:

```
Point at (5,5)
```

```
Point at (5,6)
```

REFERENCES

- [1] Core Java: Fundamentals (2021) , Cay Horstmann (Oracle Press Java).
- [2] Head First Java: A Brain-Friendly Guide (2022), Kathy Sierra, O'Reilly Media.
- [3] Java OOP Done Right: Create object oriented code you can be proud of with modern Java Paperback (2019), Mr Alan Mellor, Mellor Books.
- [4] Murach's Java Programming (5th Edition) (2017), Joe Murach, Mike Murach & Associates.
- [5]. Java for Absolute Beginners Learn to Program the Fundamentals the Java 9+ Way.
- [6]. Modern Java Recipes: Simple Solutions to Difficult Problems in Java 8 and 9 (2017), by Ken Kousen, O'Reilly Media.
- [7] Effective Java (2018), Joshua Bloch, Addison-Wesley Professional.